

Toy World – E-commerce-Database-Management-System

Product Review and Rating Management System

Introduction

The Product Review and Rating Management System is designed to store and manage customer feedback and ratings for products in the Toy World E-Commerce Database Management System. It allows customers to provide reviews and ratings for the products they purchase. The system stores review details such as customer, product, review text and review date, while the Rating table stores the rating given by customers for each product. The system can also be used to retrieve product reviews, calculate average product ratings using aggregate functions, and identify highly rated products.

Objectives

- To create Review and Rating tables for the Toy World E-Commerce Database Management System.
- To store customer feedback and ratings for different toy products.
- To retrieve product review details based on customers and products.
- To calculate the average rating of toy products using aggregate functions.
- To identify highly rated toy products based on their average ratings.

1.Database Design

1.1 Review Table

Attribute	Key Type
Review_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	-
Review_Text	-
Review_Date	-

1.2 Rating Table

Attribute	Key Type
Rating_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	-
Rating_Date	-

2. Create Review Table

```
SQL> CREATE TABLE Review (
  2     Review_ID INT PRIMARY KEY,
  3     Customer_ID INT,
  4     Product_ID INT,
  5     Rating INT,
  6     Review_Text VARCHAR2(500),
  7     Review_Date DATE,
  8     FOREIGN KEY (Customer_ID) REFERENCES Customer(ID),
  9     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
10 );
```

Table created.

3. Create Rating Table

```
SQL> CREATE TABLE Rating (
  2     Rating_ID INT PRIMARY KEY,
  3     Customer_ID INT,
  4     Product_ID INT,
  5     Rating INT,
  6     Rating_Date DATE,
  7     FOREIGN KEY (Customer_ID) REFERENCES Customer(ID),
  8     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
  9 );
```

Table created.

4. Review Data Insertion

```
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
  3  VALUES
  4  (1, 1, 101, 5, 'Very good quality toy', DATE '2026-10-01');

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
  3  VALUES
  4  (2, 2, 103, 4, 'My child loved this toy', DATE '2026-10-02');

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
  3  VALUES
  4  (3, 3, 104, 5, 'Good product and worth the price', DATE '2026-10-03');

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
  3  VALUES
  4  (4, 4, 105, 4, 'Nice toy with good quality', DATE '2026-10-04');

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
  3  VALUES
  4  (5, 5, 108, 5, 'Excellent toy for children', DATE '2026-10-05');

1 row created.
```

5. Rating Data Insertion

```
SQL> INSERT INTO Rating VALUES  
2 (1, 1, 101, 5, DATE '2026-10-01');
```

```
1 row created.
```

```
SQL>  
SQL> INSERT INTO Rating VALUES  
2 (2, 2, 103, 4, DATE '2026-10-02');
```

```
1 row created.
```

```
SQL>  
SQL> INSERT INTO Rating VALUES  
2 (3, 3, 104, 5, DATE '2026-10-03');
```

```
1 row created.
```

```
SQL>  
SQL> INSERT INTO Rating VALUES  
2 (4, 4, 105, 4, DATE '2026-10-04');
```

```
1 row created.
```

```
SQL>  
SQL> INSERT INTO Rating VALUES  
2 (5, 5, 108, 5, DATE '2026-10-05');
```

```
1 row created.
```

6. Retrieve Toy Product Review Details

```
SQL> SELECT Product_ID, Customer_ID, Review_Text, Review_Date  
2 FROM Review;
```

```

PRODUCT_ID  CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
          101             1
Very good quality toy
01-OCT-26

          103             2
My child loved this toy
02-OCT-26

          104             3
Good product and worth the price
03-OCT-26

          105             4
Nice toy with good quality
04-OCT-26

          108             5
Excellent toy for children
05-OCT-26

```

7. Calculate Average Toy Product Rating

```

SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating
2   FROM Rating
3   GROUP BY Product_ID;

```

```

PRODUCT_ID  AVERAGE_RATING
-----
          101             5
          103             4
          104             5
          105             4
          108             5

```

8. Identify Highly Rated Toy Product

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating
2   FROM Rating
3   GROUP BY Product_ID
4   HAVING AVG(Rating) >= 4;
```

PRODUCT_ID	AVERAGE_RATING
101	5
103	4
104	5
105	4
108	5

9. Product Rating Report

```
SQL> SELECT
2     P.PRODUCT_ID,
3     P.PRODUCT_NAME,
4     ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING,
5     COUNT(R.RATING_ID) AS TOTAL_RATINGS
6   FROM Product P
7  JOIN Rating R
8  ON P.PRODUCT_ID = R.PRODUCT_ID
9  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
10 ORDER BY P.PRODUCT_ID;
```

PRODUCT_ID	PRODUCT_NAME	AVERAGE_RATING	TOTAL_RATINGS
101	ABC Learning Kit	5	1
103	Remote Car	4	1
104	Chess Board	5	1
105	Football	4	1
108	Jigsaw Puzzle	5	1

Conclusion

The ToyWorld Review and Rating Management System helps store customer feedback, manage toy product ratings, calculate average ratings, and identify highly rated toy products efficiently using SQL and DBMS concepts.